

Optional: Eloquent Relationships and Queries

Connect records and control how related data is loaded.

Use the [ORM setup and Tinker examples](#).

Build with `run2-optional.sh` ; work in `orm-practice/` .

Learning Goals

- Define one-to-many and many-to-many relationships between Eloquent models.
- Avoid the N+1 query pattern with `with()` and `withCount()`.
- Use casts and query scopes to make models clearer and queries reusable.

Complete Example: One Author, Many Books

Migration for the relationship (`authors` already exists):

```
Schema::create('books', function (Blueprint $table) {  
    $table->id();  
    $table->string('title');  
    $table->foreignId('author_id')->constrained();  
    $table->timestamps();  
});
```

`author_id` is a real foreign key column. The next two slides add the Eloquent side, then use it.

Define Both Directions in the Models

```
// Author model
public function books() {
    return $this->hasMany(Book::class);
}
// Book model
public function author() {
    return $this->belongsTo(Author::class);
}
```

The foreign key column constrains the data; these methods are what let

Eloquent walk that relationship from either side.

Use the Relationship

```
$author = Author::where('name', 'Practice Author 1')->firstOrFail();  
foreach ($author->books as $book) {  
    echo $book->title;  
}
```

First Book
Second Book

`$author->books` returns a Collection, not one Book — that is why a

`foreach` works. `$book->author` returns one Author or null.

Never read

`$author->books->title` as if `books` were a single record.

Hands-On: Inspect or Build

1. In the reference app, inspect `Author` , `Book` , and their migrations.
2. Run the previous slide's query and confirm two titles appear.
3. Check each book's `author_id` matches its author's ID.

To build it yourself, use a separate fresh app, create both models and migrations, and compare with `orm-reference/` . Do not generate duplicate tables in the completed reference app.

Many Students, Many Courses

The reference creates students and courses before the pivot table.

Its `course_student` migration contains:

```
Schema::create('course_student', function (Blueprint $table) {  
    $table->foreignId('student_id')->constrained();  
    $table->foreignId('course_id')->constrained();  
    $table->timestamps();  
    $table->unique(['student_id', 'course_id']);  
});
```

The unique pair prevents duplicate enrollment in the same course.

Define Both Directions

```
// Student model
public function courses() {
    return $this->belongsToMany(Course::class)->withTimestamps();
}
// Course model
public function students() {
    return $this->belongsToMany(Student::class)->withTimestamps();
}
```

`withTimestamps()` uses the pivot's `created_at` and `updated_at`.

Add and Remove an Association

```
$student = Student::where('email', 'practice@example.com')->firstOrFail();  
$courseId = Course::where('code', 'ASE230')->firstOrFail()->id;  
$student->courses()->syncWithoutDetaching([$courseId]);  
$student->courses()->detach($courseId);
```

The course ID must exist. These operations change the pivot table.

`sync([$id1, $id2])` replaces the current set of associations, removing other links. It does not delete the course records.

Hands-On: Many-to-Many

1. Enroll one student in two courses with `syncWithoutDetaching()`.
2. Query `course_student` directly (or `DB::table('course_student')->get()`) and confirm two rows exist for that student.
3. `detach()` one course and re-check the table — confirm one row remains.
4. Try `sync([$courseId])` with a single id and confirm it replaces, not adds to, the existing rows.

Avoid the N+1 Query Pattern

```
// One query for authors, then one more per author for books
$authors = Author::all();
foreach ($authors as $author) {
    echo $author->books->count();
}

// Load authors and their books together
$authors = Author::with('books')->get();
foreach ($authors as $author) {
    echo $author->books->count();
}
```

Both loops print the same numbers — only the query count differs.

N+1 in Numbers

With 5 authors, each having some books:

Version	Queries run
<code>Author::all()</code> , then <code>\$author->books</code> per author	1 (authors) + 5 (one per author) = 6
<code>Author::with('books')->get()</code>	1 (authors) + 1 (all their books) = 2

To see this yourself:

```
DB::enableQueryLog();
DB::flushQueryLog(); // Reset before EACH version
// Run either version from the previous slide.
count(DB::getQueryLog());
DB::disableQueryLog();
```

Count Without Loading Every Book

```
$authors = Author::withCount('books')->get();  
foreach ($authors as $author) {  
    echo $author->books_count;  
}
```

`withCount()` adds a count subquery to the author query — it never loads the book rows themselves. Use `with()` when you need the records; use `withCount()` when you only need the number.

Cast Stored Values

Inside a Student model whose table has these columns:

```
protected function casts(): array
{
    return ['year' => 'integer', 'is_active' => 'boolean'];
}
```

Casting controls how a value comes back out of the model — it does not create columns or replace input validation. Nullable attributes can still be null.

See the Cast Take Effect

```
$student = Student::where('email', 'practice@example.com')->firstOrFail();  
gettype($student->year);           // integer  
gettype($student->is_active);      // boolean
```

Without a cast, the returned type can depend on the database driver.

A cast makes the model's PHP type explicit. It does not change the column type or replace validation.

Reuse a Query with a Scope

Inside the Student model:

```
public function scopeInYear($query, $year)
{
    return $query->where('year', $year);
}
```

Use `Student::inYear(2)->orderBy('name')->get()`.

The scope adds a condition; `get()` executes the query.

See the Scope's SQL

```
Student::inYear(2)->toSql();
```

```
select * from `students` where `year` = ?
```

`toSql()` shows the query without running it — a quick way to confirm a scope added exactly the condition you expected before you call `get()`.

Hands-On: Casting and Scopes

1. Inspect the reference Student model's casts and check both PHP types.
2. Run `Student::inYear(2)->toSql()` and inspect the condition.
3. Chain the included scopes: `Student::inYear(2)->active()->get()` .

If you edit model code, exit and reopen Tinker to load the changes.

Confirm the query returns the active, year-2 practice student.